# DepSolve CLI

## Project Summary

### Package Dependency Resolver CLI

#### Nama Produk

**DepSolve CLI**

---

# Deskripsi Produk

**DepSolve CLI** adalah aplikasi command-line berbasis Python yang digunakan untuk melakukan manajemen package dan dependency secara otomatis seperti pada package manager modern (`npm`, `pip`, atau `apt`), namun dalam versi simulasi edukatif yang berfokus pada implementasi algoritma dan struktur data.

Aplikasi ini memungkinkan pengguna untuk:

- menginstal package,
- menghapus package,
- mendeteksi konflik dependency,
- menampilkan dependency tree,
- melakukan validasi versi package,
- serta mencegah circular dependency.

Seluruh data package dan dependency disimpan menggunakan database MySQL yang diakses melalui phpMyAdmin.

Fokus utama aplikasi ini adalah implementasi:

- graph,
- hash map,
- stack/queue,

- traversal algorithm,
- topological sorting,
- dan cycle detection.

Aplikasi dikembangkan menggunakan bahasa Python dengan antarmuka berbasis CLI (Command Line Interface).

---

# Latar Belakang

Pada sistem modern, sebuah package sering bergantung pada package lain untuk dapat berjalan dengan baik. Semakin kompleks dependency yang dimiliki suatu aplikasi, semakin sulit proses instalasi dan pengelolaannya.

Kesalahan dependency dapat menyebabkan:

- konflik versi,
- circular dependency,
- package rusak,
- atau dependency tidak kompatibel.

Oleh karena itu dibutuhkan sistem resolver yang mampu:

- membaca hubungan antar package,
- menentukan urutan instalasi,
- dan mendeteksi konflik secara otomatis.

Project ini dibuat sebagai simulasi package manager modern sekaligus implementasi nyata mata kuliah Struktur Data dan Algoritma.

---

# Tujuan Proyek

1. Mengimplementasikan struktur data graph dalam studi kasus nyata.
2. Mengimplementasikan algoritma traversal dan dependency resolution.
3. Membuat aplikasi CLI non-sistem-informasi yang kompleks dan bersifat engineering.

4. Mengintegrasikan Python dengan MySQL.
5. Mensimulasikan cara kerja package manager modern.

---

# Fitur Utama

## 1. Install Package

Menginstal package beserta seluruh dependency yang dibutuhkan.

Contoh:

```
depsolve install flask
```

---

## 2. Remove Package

Menghapus package jika tidak digunakan package lain.

```
depsolve remove numpy
```

---

## 3. Dependency Tree Visualization

Menampilkan struktur dependency berbentuk tree.

```
depsolve tree django
```

Output:

```
django
├── pytz
├── sqlparse
└── asgiref
```

---

## 4. Conflict Detection

Mendeteksi konflik dependency antar package.

```
depsolve conflicts
```

---

## 5. Circular Dependency Detection

Mendeteksi dependency melingkar menggunakan DFS.

Contoh:

```
A → B → C → A
```

---

## 6. Installed Package List

Menampilkan seluruh package yang telah terinstal.

```
depsolve list
```

---

## 7. Version Validation

Validasi kompatibilitas versi package.

Contoh:

```
flask membutuhkan werkzeug >=2.0
```

---

# Struktur Data yang Digunakan

## 1. Graph

Digunakan untuk merepresentasikan hubungan dependency antar package.

Contoh:

```
Flask → Werkzeug
Flask → Jinja2
```

---

## 2. Hash Map (Dictionary)

Digunakan untuk:

- pencarian package cepat,
- penyimpanan cache dependency,
- validasi package.

---

## 3. Stack

Digunakan pada:

- DFS traversal,
- cycle detection.

---

## 4. Queue

Digunakan untuk:

- BFS traversal,
- proses instalasi berurutan.

---

# Algoritma yang Digunakan

## DFS (Depth First Search)

Untuk:

- traversal dependency,
- cycle detection.

---

## BFS (Breadth First Search)

Untuk:

- eksplorasi dependency level-by-level.

---

## Topological Sort

Menentukan urutan instalasi package yang benar.

Contoh:

```
Install:
1. Werkzeug
2. Jinja2
3. Flask
```

---

## Cycle Detection

Mendeteksi circular dependency pada graph.

---

# Teknologi yang Digunakan

| Teknologi | Fungsi |
|---|---|
| Python | Bahasa utama aplikasi |
| MySQL | Database package |
| phpMyAdmin | Database management |
| Click / Argparse | CLI framework |
| SQLAlchemy | ORM database |
| Rich | Tampilan terminal interaktif |

---

# Rancangan Database

## Table: packages

| Field | Tipe |
|---|---|
| id | INT |
| name | VARCHAR |
| version | VARCHAR |

---

## Table: dependencies

| Field | Tipe |
|---|---|
| id | INT |
| package_id | INT |
| dependency_id | INT |
| min_version | VARCHAR |

---

## Table: installed_packages

| Field | Tipe |
|---|---|
| id | INT |
| package_id | INT |
| install_date | DATETIME |

---

# Contoh Alur Sistem

## Install Flask

```
User → install flask
```

System:

1. cek package flask
2. baca dependency

3. traversal graph
4. validasi cycle
5. topological sort
6. install dependency dulu
7. install flask

---

# Kelebihan Project

- Bukan sistem informasi biasa
- Sangat kuat di struktur data
- Ada implementasi graph nyata
- CLI engineering style
- Portfolio software engineering bagus
- Mirip teknologi industri asli

---

# Tingkat Kompleksitas

## Kompleksitas:

**Menengah – Tinggi**

Melibatkan:

- graph traversal,
- dependency resolution,
- recursion,
- sorting dependency,
- database relation,
- serta simulasi package manager nyata.

---

# Potensi Pengembangan

- semantic versioning (`^1.2.0`)
- online repository simulation
- package publishing
- auto update
- dependency locking
- local cache system
- package integrity hash
- multi-user repository